Divya – User & Authentication Management Module

Oct 10, 2026 · @Naruti

Module 1 of 4, owned by Divya: accounts, roles, locations, dashboards and pharmacy approval, plus the shared foundation the other three modules build on.

1. Module description and objectives

Divya's module gives every actor of the Emergency Medicine Availability Network (patient, hospital, pharmacy, delivery partner and administrator) a secure account, a role, a saved location and a dashboard. It also decides which pharmacies may operate: a pharmacy becomes usable only after an administrator approves it. It is the foundation of the project, because it supplies the shared Java package `common` (response envelope, error handling, security, contract interfaces) and the shared UI assets (theme, navbar, API helper) that Leelavathy, Deepika and Anusuya build on.

Technology: HTML, CSS, JavaScript, Bootstrap 5 (frontend); Java 17 with Spring Boot 3, Spring Data JPA, Spring Security with BCrypt, Maven (backend); MySQL 8 with Flyway migrations (database).

Objectives

- Register and authenticate the four public roles safely (BCrypt, session login, CSRF protection).
- Enforce role-based access for every page and API in the whole application.
- Store and update profile and location data, including pharmacy coordinates.
- Let the admin approve, reject or suspend pharmacies and manage users.
- Give each role its own dashboard with links into the other three modules.
- Publish the shared foundation on day one so the other members can start.

2. Features

- Registration for patient, hospital, pharmacy and delivery partner, each with its own form and validation.
- Login, logout, session timeout (30 minutes), change password, generic login error message.
- Roles `PATIENT`, `HOSPITAL`, `PHARMACY`, `DELIVERY`, `ADMIN` with page and API guards.

- Profile view and edit; location management (address, city, pincode, latitude, longitude) with a Use my current location button.
- Pharmacy approval workflow: `PENDING`, `APPROVED`, `REJECTED`, `SUSPENDED`, with a mandatory reason for reject and suspend.
- Admin user management: search, activate or deactivate, verify delivery partners.
- Separate dashboards for the five roles.
- Shared foundation: `ApiResponse` envelope, `GlobalExceptionHandler`, `SecurityConfig`, contract interfaces, `GeoUtils` (Haversine distance), `MapLinks` (Google Maps Directions URL), `PhoneUtils`, `theme.css`, `api.js`, `ui.js`, `auth-guard.js`.
- Demo data loader: admin, patient, hospital, five pharmacies, two delivery partners (fictional names, fake phone numbers).

3. Workflow

1. A visitor opens `register.html`, chooses a role and submits the matching form. The backend validates the input, hashes the password and creates the `users` row plus the role profile row.
2. Patient, hospital and delivery accounts are active immediately. A pharmacy is created as `PENDING`; it can log in but sees an Awaiting approval banner, and every pharmacy action in Leelavathy's and Anusuya's modules returns `PHARMACY_NOT_APPROVED`.
3. The admin opens `admin/pharmacy-approvals.html`, checks licence number, address and map location, then approves or rejects. Approval publishes `PharmacyApprovedEvent` (Leelavathy's module creates the default pharmacy settings) and sends an in-app notification through `NotificationPublisher`.
4. The user logs in. The backend creates the session and returns the role and a redirect URL; `auth-guard.js` opens the correct dashboard.
5. The user edits profile and location. A pharmacy's saved coordinates are the ones used by Deepika's search for distance.
6. Logout ends the session and clears the CSRF cookie; protected pages redirect to `login.html`.

4. Frontend requirements

Use HTML, CSS, JavaScript and Bootstrap 5.3 with Bootstrap Icons. Navy `#102A43` is primary, Teal `#0D9488` secondary, Red `#DC2626` for emergency alerts and destructive actions, backgrounds `#F1F5F9` and white, font Inter with a sans-serif fallback. Divya

publishes these as `theme.css` variables, and the other three modules reuse them. Every page needs a loading spinner, an empty state, inline validation messages and success or error toasts; every button and form must call a defined API or open a defined page.

Page	Purpose and active controls
`index.html`	Landing page with hero, three feature cards, Login and Register buttons, demo-data disclaimer
`register.html`	Four role cards linking to the four forms
`register-patient.html`	Name, email, phone, password, confirm password, date of birth, blood group, city, location. Register calls `POST /auth/register/patient`
`register-hospital.html`	Hospital name, registration number, emergency phone, address, location. Calls `POST /auth/register/hospital`
`register-pharmacy.html`	Pharmacy name, licence number, address, location with Use my current location. Success screen says approval is pending. Calls `POST /auth/register/pharmacy`
`register-delivery.html`	Name, phone, vehicle type and number, licence number, city. Calls `POST /auth/register/delivery-partner`
`login.html`	Email, password, show password toggle, Login (`POST /auth/login`), red alert on failure
`profile.html`	Tabs Details, Location, Security. Save (`PUT /users/me`), Update location (`PUT /users/me/location`), Change password (`PUT /auth/change-password`)
`dashboard/patient.html`	Cards: Find medicine (to `search.html`), My requests (`GET /requests/summary`), Notifications, Profile
`dashboard/hospital.html`	Same as patient plus Create emergency request
`dashboard/pharmacy.html`	Approval status banner, Inventory summary (`GET /inventory/summary`), Incoming requests (`GET /requests/summary`), Settings
`dashboard/delivery.html`	Availability link, Open jobs, History

Page	Purpose and active controls
`dashboard/admin.html`	Counters from `GET /admin/summary`; pending pharmacies shown with a red badge
`admin/users.html`	Table with search, role filter, Activate or Deactivate, Verify partner
`admin/pharmacy-approvals.html`	Tabs Pending, Approved, Rejected; Approve, Reject (modal, reason required), Suspend

Shared assets Divya must publish first: `assets/css/theme.css`, `assets/js/api.js` (fetch wrapper that sends the CSRF header and unwraps the envelope), `ui.js` (role-aware navbar, spinner, toast, empty-state helper, notification bell), `auth-guard.js`. She also creates empty stub pages (with the navbar) for every page owned by the other three modules so links never break. Dashboard cards that call another module must show a placeholder, not an error, if that endpoint is not available yet.

Navbar links by role: PATIENT and HOSPITAL: `/search.html`, `/request/new.html`, `/request/my-requests.html`, `/notifications.html`, `/profile.html`. PHARMACY: `/pharmacy/inventory.html`, `/pharmacy/settings.html`, `/pharmacy/requests.html`, `/notifications.html`, `/profile.html`. DELIVERY: `/delivery/jobs.html`, `/delivery/history.html`, `/notifications.html`, `/profile.html`. ADMIN: `/admin/users.html`, `/admin/pharmacy-approvals.html`, `/admin/medicines.html`, `/admin/requests.html`. Public: `/medicine-catalogue.html`.

5. Spring Boot backend requirements

- Authentication with server-side HTTP session, BCrypt (strength 10), CSRF through `CookieCsrfTokenRepository`, session fixation protection, 30-minute timeout.
- `SecurityConfig`: public paths (`/api/v1/auth/login`, `/api/v1/auth/register/**`, `GET /api/v1/medicines/**`, static files), role rules, JSON bodies for 401 and 403.
- Registration and profile services for the four roles; pharmacy approval state machine (`PENDING` to `APPROVED` or `REJECTED`; `APPROVED` to `SUSPENDED` and back).
- Publish `PharmacyApprovedEvent` and `PharmacyRejectedEvent`; call `NotificationPublisher` (fallback bean logs only).
- Implement `CurrentUserProvider`, `PharmacyLookup` (bounding-box query plus Haversine in `findApprovedWithin`) and `UserLookup` (`activeDeliveryPartnerIds` returns verified, active partners).

- Shared code: `ApiResponse`, `PageResponse`, `ErrorCode`, `BusinessException`, `GlobalExceptionHandler`, enums, view DTOs, `GeoUtils.haversineKm`, `MapLinks.directionsUrl`, `PhoneUtils`.
- Flyway versions `V100` to `V199`; `DemoDataLoader` (profile `demo`, order 100) creating demo users and five demo pharmacies.

```
com.emergencymed
  EmergencyMedicineApplication.java
  common/
    api/        ApiResponse, PageResponse, ErrorItem
    config/     SecurityConfig, WebConfig, JacksonConfig
    contract/   CurrentUserProvider, PharmacyLookup, UserLookup,
MedicineLookup,
                InventoryQueryService, InventoryService,
PharmacySettingsLookup,
                DeliveryAvailabilityQuery, NotificationPublisher, view DTOs
    enums/      Role, ApprovalStatus, DataSource, NotificationType,
DeliveryStatus
    events/     PharmacyApprovedEvent, PharmacyRejectedEvent
    exception/  BusinessException, ErrorCode, GlobalExceptionHandler
    util/       GeoUtils, MapLinks, PhoneUtils
  auth/
    controller/ AuthController, UserController, PharmacyAccountController,
                AdminUserController, AdminPharmacyController
    service/    AuthService, ProfileService, PharmacyApprovalService,
contract implementations
    repository/ UserRepository, PatientProfileRepository,
HospitalProfileRepository,
                PharmacyRepository, DeliveryPartnerProfileRepository
    entity/     User, PatientProfile, HospitalProfile, Pharmacy,
DeliveryPartnerProfile
    dto/        request and response records
    security/   CustomUserDetailsService, CurrentUserProviderImpl
    demo/       DemoDataLoader
src/main/resources
  db/migration/ V100__m1_users_profiles.sql,
V110__m1_pharmacies_delivery_profiles.sql
  static/assets/ css/theme.css, js/api.js, js/ui.js, js/auth-guard.js
  static/        index.html, login.html, register*.html, profile.html,
dashboard/, admin/
```

6. MySQL database tables (owned by Divya)

Five tables, Flyway files `V100__m1_users_profiles.sql` and `V110__m1_pharmacies_delivery_profiles.sql`. Naming: plural `snake_case` tables, `BIGINT` auto-increment IDs, coordinates `DECIMAL(9,6)`, timestamps in UTC.

`users`: `id` BIGINT PK; `full_name` VARCHAR(100); `email` VARCHAR(120) UNIQUE; `phone` VARCHAR(15); `password_hash` VARCHAR(100); `role` VARCHAR(20); `is_active` TINYINT(1) default 1; `last_login_at` DATETIME; `created_at`, `updated_at` DATETIME.

`patient_profiles`: `id` PK; `user_id` FK to `users.id` UNIQUE; `date_of_birth` DATE; `gender` VARCHAR(10); `blood_group` VARCHAR(5); `address` VARCHAR(255); `city` VARCHAR(60); `pincode` CHAR(6); `latitude`, `longitude` DECIMAL(9,6); `emergency_contact_name`, `emergency_contact_phone`.

`hospital_profiles`: `id` PK; `user_id` FK to `users.id` UNIQUE; `hospital_name` VARCHAR(150); `registration_no` VARCHAR(50) UNIQUE; `hospital_type` VARCHAR(30); `emergency_phone` VARCHAR(15); `address`, `city`, `pincode`, `latitude`, `longitude`.

`pharmacies`: `id` PK; `user_id` FK to `users.id` UNIQUE; `name` VARCHAR(150); `license_no` VARCHAR(50) UNIQUE; `gst_no` VARCHAR(20) NULL; `address` VARCHAR(255); `city`; `pincode`; `latitude`, `longitude` DECIMAL(9,6) NOT NULL; `approval_status` VARCHAR(15) default `PENDING`; `rejection_reason` VARCHAR(255) NULL; `approved_by_user_id` FK to `users.id` NULL; `approved_at` DATETIME NULL; `is_active` TINYINT(1); `is_demo` TINYINT(1); `created_at`, `updated_at`.

`delivery_partner_profiles`: `id` PK; `user_id` FK to `users.id` UNIQUE; `vehicle_type` VARCHAR(20); `vehicle_number` VARCHAR(20); `driving_license_no` VARCHAR(30); `service_city` VARCHAR(60); `is_verified` TINYINT(1) default 0; `created_at`.

Relationships: `users` 1 to 1 with each profile table (one profile per role); `users` 1 to 1 `pharmacies`; `pharmacies.approved_by_user_id` many to 1 `users`. Indexes: `users(email)`, `users(role, is_active)`, `pharmacies(approval_status)`, `pharmacies(latitude, longitude)`. No cascading deletes: users and pharmacies are deactivated, never deleted.

7. REST APIs

Base path `/api/v1`. Every response uses the shared envelope `{success, message, data, errors[], timestamp}`.

Method	Path	Role	Purpose
POST	`/auth/register/patient`	Public	Create patient
POST	`/auth/register/hospital`	Public	Create hospital
POST	`/auth/register/pharmacy`	Public	Create pharmacy (`PENDING`)
POST	`/auth/register/delivery-partner`	Public	Create delivery partner
POST	`/auth/login`	Public	Login; returns `{userId, fullName, role, approvalStatus, redirectUrl}`
POST	`/auth/logout`	Any	End session
GET	`/auth/me`	Any	Current user summary
PUT	`/auth/change-password`	Any	Old and new password
GET	`/users/me`	Any	User plus role profile
PUT	`/users/me`	Any	Update name, phone, role profile fields
PUT	`/users/me/location`	Any	Update address, latitude, longitude (also pharmacy coordinates)
GET	`/pharmacies/me`	PHARMACY	Own pharmacy and approval status
PUT	`/pharmacies/me`	PHARMACY	Update name and address (licence number locked)
GET	`/admin/summary`	ADMIN	Dashboard counters
GET	`/admin/users`	ADMIN	List with `role`, `active`, `q`, paging
PATCH	`/admin/users/{id}/status`	ADMIN	Body `{active}`
PATCH	`/admin/delivery-partners/{userId}/verify`	ADMIN	Mark verified
GET	`/admin/pharmacies`	ADMIN	Filter by `status`
GET	`/admin/pharmacies/{id}`	ADMIN	Details

Method	Path	Role	Purpose
PATCH	`/admin/pharmacies/{id}/approve`	ADMIN	Approve
PATCH	`/admin/pharmacies/{id}/reject`	ADMIN	Body `{reason}` required
PATCH	`/admin/pharmacies/{id}/suspend`	ADMIN	Suspend with reason

8. Integration requirements

What Divya provides and consumes

- Provides to everyone: `common` package, `CurrentUserProvider`, `PharmacyLookup`, `UserLookup`, `GeoUtils`, `MapLinks`, `PhoneUtils`, events, theme and shared JavaScript, all navbar links and stub pages. Merge this first (phase 1).
- Consumes (optional, with log-only fallback): `NotificationPublisher` from Anusuya's module when approving or rejecting a pharmacy, and the summary endpoints `GET /inventory/summary` (Leelavathy) and `GET /requests/summary` (Anusuya) for dashboard cards.
- Hands over: Leelavathy listens to `PharmacyApprovedEvent`; Deepika calls `PharmacyLookup.findApprovedWithin`; Anusuya calls `UserLookup.activeDeliveryPartnerIds`. Page links: dashboards link to `/search.html` (Deepika), `/pharmacy/inventory.html` (Leelavathy), `/request/*.html` and `/notifications.html` (Anusuya).

Shared conventions (identical in all four module documents)

- Base path `/api/v1`. Every response is `{success, message, data, errors[{field,message}], timestamp}`. Lists use `data = {content, page, size, totalElements, totalPages}` with params `page` (from 0), `size` (default 10, maximum 50) and `sort`.
- Roles: `PATIENT`, `HOSPITAL`, `PHARMACY`, `DELIVERY`, `ADMIN`. The user and pharmacy ID always come from the session through `CurrentUserProvider`, never from the request body.
- Naming: tables plural `snake_case`, entities singular `PascalCase`, JSON `camelCase`, enums `UPPER_SNAKE` stored as strings, `BIGINT` IDs, money `DECIMAL(10,2)` in INR, coordinates `DECIMAL(9,6)`, timestamps in UTC.
- Login is a server-side session; every `POST`, `PUT`, `PATCH`, `DELETE` carries the CSRF header `X-XSRF-TOKEN` (added by `api.js`).

- Stock: `availableQuantity = quantity - reservedQuantity`. Inventory rows are `SAMPLE` or `LIVE`; sample data is never presented as real-time stock.
- Prescription-only medicines need a prescription verified by the pharmacy before a request is accepted.

HTTP	`errorCode`	When
400	`VALIDATION_ERROR`	Bean validation or bad parameter
401	`UNAUTHORIZED`	Not logged in or session expired
403	`FORBIDDEN`	Wrong role or another user's data
403	`PHARMACY_NOT_APPROVED`	Pharmacy action before approval
404	`NOT_FOUND`	Missing record
409	`DUPLICATE_RESOURCE`	Email, licence number or medicine already exists
409	`INSUFFICIENT_STOCK`	Reservation larger than available stock
409	`INVALID_STATE_TRANSITION`	Status change not allowed
409	`NO_DELIVERY_PARTNER`	Delivery not possible; offer self-pickup
422	`PRESCRIPTION_REQUIRED`	Prescription-only item without a verified prescription
500	`INTERNAL_ERROR`	Unexpected; generic message only

Shared database design (one MySQL database `emergency_med_db`, 19 tables)

Owner	Module	Tables	Flyway range
Divya	User & Authentication	`users`, `patient_profiles`, `hospital_profiles`, `pharmacies`, `delivery_partner_profiles`	`V100` to `V199`
Leelavathy	Medicine & Inventory	`medicine_categories`, `medicines`, `pharmacy_settings`, `pharmacy_inventory`, `inventory_transactions`	`V200` to `V299`
Deepika	Medicine Search & Availability	`search_history`, `contact_logs`	`V300` to `V399`

Owner	Module	Tables	Flyway range
Anusuya	Emergency Request & Notification	`emergency_requests`, `request_items`, `prescriptions`, `request_status_history`, `delivery_availability`, `delivery_assignments`, `notifications`	`V400` to `V499`

Only the owner reads or writes a table directly. Other modules use the Java interfaces below. No `ON DELETE CASCADE` across modules.

Interface (`com.emergencymed.common.contract`)	Implemented by	Used by	Methods
`CurrentUserProvider`	Divya	all	`Long userId()`, `Role role()`, `Optional<Long> pharmacyId()`, `boolean isAuthenticated()`
`PharmacyLookup`	Divya	Leelavathy, Deepika, Anusuya	`findApproved(id)`, `findApprovedWithin(lat, lng, radiusKm)`, `pharmacyIdForUser(userId)`
`UserLookup`	Divya	Anusuya	`findById(userId)`, `activeDeliveryPartnerIds()`
`MedicineLookup`	Leelavathy	Deepika, Anusuya	`findById(id)`, `searchByName(q, limit)`
`InventoryQueryService`	Leelavathy	Deepika	`findOffers(medicineId, qty, pharmacyIds)`, `findAlternatives(medicineId, pharmacyIds)`
`InventoryService`	Leelavathy	Anusuya	`reserve`, `commit`, `release`, `restore` (pharmacyId or inventoryId, qty, refCode)
`PharmacySettingsLookup`	Leelavathy	Deepika, Anusuya	`find(pharmacyId)`
`DeliveryAvailabilityQuery`	Anusuya	Deepika	`statusFor(pharmacyId)`: `AVAILABLE`, `BUSY`, `UNAVAILABLE`
`NotificationPublisher`	Anusuya	Divya, Leelavathy, Anusuya	`publish(userId, type, title, message, requestId)`

Every interface a module consumes needs a `@ConditionalOnMissingBean` fallback so that module runs alone before integration.

9. Validation, exception handling and security

Field	Rule
Email	Valid, unique, lower-cased before saving
Phone	`^[6-9][0-9]{9}$`
Password	8 to 64 characters with upper, lower, digit and special character; confirm must match
Pincode	6 digits
Latitude and longitude	-90 to 90 and -180 to 180; required for pharmacies
Licence and registration number	Required, unique, 5 to 50 characters
Reject or suspend reason	Required, at least 10 characters

- Duplicate email or licence number returns 409 `DUPLICATE_RESOURCE`. Validation errors list the field and message.
- Failed login gives the same message for an unknown email and a wrong password; a deactivated account gets 403.
- BCrypt strength 10; password hashes never appear in responses or logs; session fixation protection; `HttpOnly` cookie; CSRF enabled; 30-minute timeout.
- Admin endpoints require `ROLE_ADMIN`; an admin cannot deactivate their own account.
- All input is validated on the server even when the browser form validates it. `GlobalExceptionHandler` never returns a stack trace.

10. Testing scenarios and expected output

Scenario	Steps	Expected
Patient registers	Submit a valid form	201; user and profile rows exist; password stored as a BCrypt hash (starts with `$2`)
Duplicate email	Register twice with the same email	409 `DUPLICATE_RESOURCE` with a field error on email

Scenario	Steps	Expected
Weak password	Password `abc123`	400 listing the missing rules
Pharmacy pending	Register a pharmacy, log in, open inventory (Leelavathy)	Dashboard shows the pending banner; API returns 403 `PHARMACY_NOT_APPROVED`
Admin approves	Approve the pending pharmacy	Status `APPROVED`, `approved_at` set, notification sent, default settings row created by Leelavathy's module
Reject needs a reason	Reject with an empty reason	400 `VALIDATION_ERROR`
Wrong role	Patient calls `GET /admin/users`	403 `FORBIDDEN` in the standard envelope
Not logged in	Open `dashboard/patient.html` without a session	Redirect to `login.html`; API returns 401
CSRF	`POST` without the CSRF header	403
Logout	Log out, press Back, refresh	Session ended; redirect to login
Location update	Use my current location, save	Coordinates saved and returned by `GET /users/me`
Distance helper	`GeoUtils.haversineKm` for Adyar and T. Nagar	About 5 km